KU LEUVEN

FACULTEIT INDUSTRIËLE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Labo 1

Labo Geavanceerde Computerarchitectuur

Kjell NAESSENS
Beau FORREZ

Labo verantwoordelijke: T. Feys

Academiejaar 2025-2026

Hoofdstuk 1

Verslag

1.1 Context

Het doel van deze opdracht was het implementeren van een eenvoudige parallelle bewerking op een GPU met behulp van CUDA. De specifieke taak was het omkeren (flippen) van een array van gehele getallen. Hierbij kunnen we spelen met het aantal blokken en threads om de berekening parallel uit te voeren en de efficiëntie van GPU-verwerking te ervaren.

$$[0,1,2,3,4,5,6,7,8] \rightarrow [8,7,6,5,4,3,2,1,0]$$

1.2 Methodologie

De aanpak bestond uit de volgende stappen:

1. **Initialisatie van data:** Een array van N gehele getallen werd aangemaakt op de host (CPU) en gevuld met opeenvolgende waarden van 0 tot $N-1$.
2. **Allocatie van geheugen op de GPU:** Twee arrays van gelijke grootte werden aangemaakt in het device-geheugen, een voor de input en een voor de output.
3. **Data-overdracht:** De input-array werd van de host naar het device gekopiëerd met behulp van `cudaMemcpy`.
4. **Kernel-ontwikkeling:** Een CUDA-kernel `flip` werd geschreven waarbij elke thread verantwoordelijk is voor het kopiëren van een element van de input-array naar de juiste positie in de output-array zodat de array volledig omgekeerd wordt.
5. **Configuratie van threads en blocks:** Het aantal threads per block wordt bepaald op basis van de array-lengte, zodat elk element door exact één thread verwerkt wordt. Dit maakt gebruik van parallelisme.

6. **Resultaat terughalen:** Nadat de kernel was uitgevoerd, werd de output-array teruggekopieerd naar de host met `cudaMemcpy`.

7. **Validatie:** De originele en geflipte arrays werden afgedrukt om visueel te controleren of de operatie correct was uitgevoerd.

1.3 Observaties en resultaten

De implementatie gaf de volgende resultaten voor een voorbeeldarray van lengte 9:

In	Out
0	8
1	7
2	6
3	5
4	4
5	3
6	2
7	1
8	0

De array werd correct omgekeerd door de GPU, wat aantoont dat het gebruik van meerdere threads en blocks succesvol was. De uitvoering op de GPU zou merkbaar sneller moeten zijn bij grotere arrays in vergelijking met een sequentiële uitvoering op de CPU, wat het voordeel van parallel programmeren illustreert.

1.4 Besluit

Deze opdracht heeft duidelijk gemaakt hoe CUDA kan worden gebruikt om parallelle bewerkingen op arrays uit te voeren. Het belangrijkste inzicht was dat:

- GPU-programmering het mogelijk maakt om eenvoudige taken zoals array-flipping efficiënt te paralleliseren.
- Correcte configuratie van blocks en threads cruciaal is om alle array-elementen te verwerken.
- Het migreren van data tussen host en device zorgvuldig moet gebeuren om correcte resultaten te garanderen.

In het algemeen laat deze oefening zien hoe parallel programmeren op GPU's werkt en bereidt het voor op complexere berekeningen en oefeningen.

Hoofdstuk 2

Appendix

2.1 Code

```
1
2  #include <iostream>
3  #include <cuda_runtime.h>
4
5
6  // CUDA kernel to flip an array
7  __global__ void flip(int *a, int *b, int n) {
8      int idx = threadIdx.x + blockIdx.x * blockDim.x; // know what thread we are
           on
9      if (idx < n) { // only execute on the correct element
10         b[n-idx-1] = a[idx]; // flip element
11     }
12 }
13
14 int main() {
15     const int N = 10; // length of array
16     int a[N], b[N]; // input array, output array
17     int *d_a, *d_b; // allocated device memory
18
19     // Initialize array
20     for (int i = 0; i < N; i++) {
21         a[i] = i;
22     }
23
24     // Allocate device memory
25     cudaMalloc((void**)&d_a, N * sizeof(int));
26     cudaMalloc((void**)&d_b, N * sizeof(int));
27
28     // Copy data from host to device
29     cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
30
31     // Launch kernel (1 block of N threads)
```

```
32      flip<<<1, N>>>(d_a, d_b, N);
33
34      // Copy result back to host
35      cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);
36
37      // Print results
38      std::cout << "Result: ";
39      for (int i = 0; i < N; i++) {
40          std::cout << b[i] << " ";
41      }
42      std::cout << std::endl;
43
44      // Free device memory
45      cudaFree(d_a);
46      cudaFree(d_b);
47
48      return 0;
49  }
```

Listing 2.1: C++ code om een array te flippen